NAME: Kania Punit Chetan
ROLL-NO:276
DIV=D
SYBCA(Regular)-SEM-3

1 . Create Student Table with appropriate constraints. STUDENT(sno number primary key, sname text(20), age number, total_marks number) write python programs to perform following task: 1) store the table data into a dataframe and display the dataframe. 2) List out top three records from the dataframe 3) Display all records from dataframe whose age is not less than 18. 4) Display age of student whose sno is 5. (use loc() and iloc() function)

```
import pandas as pd

data={
  'sno':[1,2,3,4,5],
  'sname':['krish','veer','sujal','janvi','ravi'],
  'age':[19,16,11,22,25],
  'total_marks':[85,55,92,65,100]
}
df=pd.DataFrame(data) print("DataFrame:")  print(df)

top_three=df.head(3) print("\n Top three records:")
print(top_three) above_18=df[df['age']>=18] print("\n
Records
1
with age not less than 18:") print(above_18)
row_loc=df.loc[df['sno']==5 ,'age'].values[0]
row_iloc=df.iloc[df.index[df['sno']==5],df.columns
.get_loc('age')].values[0] print("\nage of student with sno 5
(loc):",row_loc) print("\nage of student with sno 5
(iloc):",row_iloc)
```

NAME: Kania Punit Chetan
ROLL-NO:276
DIV=D
SYBCA(Regular)-SEM-3

OUTPUT

```
>>>
======= RESTART: D:\sqlite3\sqlite-tools-win32-x86-3420000\journal(1).py =======
DataFrame:
   sno  sname  age  total_marks
0    1  krish   19           85
1    2   veer   16           55
2    3  sujal   11           92
3    4  janvi   22           65
4    5   ravi   25          100

 Top three records:
   sno  sname  age  total_marks
0    1  krish   19           85
1    2   veer   16           55
2    3  sujal   11           92

 Records with age not less than 18:
   sno  sname  age  total_marks
0    1  krish   19           85
3    4  janvi   22           65
4    5   ravi   25          100

age of student with sno 5 (loc): 25

age of student with sno 5 (iloc): 25
>>>
```

2. Create following table and store an`y five records:
Employee(eno number primary key, Ename text(20),designation text(10),basic number , da number, gross_salary number) write python programs to perform following tasks: 1) Store the table data into dataframe and display the dataframe. 2) Sort the dataframe based used on gross salary and List out bottom two record from the dataframe. 3) Display all records from dataframe whose gross Display gross salary is more than 25000 . 4) Display gross salary of employee whose eno is 4.

Ans 2 : import pandas as pd

import numpy as np dict={

```
'eno':[1,2,3,4,5],

  'Ename':['krish','veer','sujal','janvi','ravi'],

  'designation':['manager','account','technician','
  manager','proffesor'],

  'basic':[15000,20000,70000,40000,50000],


  'da':[1500,2000,7000,4000,5000],

  'gross_salary':[17500,22000,77000,44000,55000
  ]
  }
dataframe=pd.DataFrame(dict)                print('Ans        1.')

print(dataframe)

sorted=dataframe.sort_values(by='gross_salary')

bottom_two=sorted.tail(2)              print('\nAns              2.')

print(bottom_two)

salary_more_than_25000=dataframe[dataframe['
gross_salary'] >= 25000]  print('\nAns 3.')
print(salary_more_than_25000)

employee_4=dataframe.loc[dataframe['eno'] ==

4,'gross_salary'].values[0] print('\nAns 4.')

print(employee_4)
```

NAME: Kania Punit Chetan
ROLL-NO:276
DIV=D
SYBCA(Regular)-SEM-3

OUTPUT

```
>>>
======= RESTART: D:\sqlite3\sqlite-tools-win32-x86-3420000\journal(2).py =======

DataFrame:
   eno  ename designation  basic    da  gross_salary
0    1  krish     manager  15000  1500         17500
1    2   veer  accountant  20000  2000         22000
2    3  sujal  technician  70000  7000         77000
3    4  janvi     manager  40000  4000         44000
4    5   ravi   proffesor  50000  5000         55000

sort records by gross salary:
   eno  ename designation  basic    da  gross_salary
0    1  krish     manager  15000  1500         17500
1    2   veer  accountant  20000  2000         22000
3    4  janvi     manager  40000  4000         44000
4    5   ravi   proffesor  50000  5000         55000
2    3  sujal  technician  70000  7000         77000

bottom 2 records:
   eno  ename designation  basic    da  gross_salary
4    5   ravi   proffesor  50000  5000         55000
2    3  sujal  technician  70000  7000         77000

gross salary more than 25000:
   eno  ename designation  basic    da  gross_salary
2    3  sujal  technician  70000  7000         77000
3    4  janvi     manager  40000  4000         44000
4    5   ravi   proffesor  50000  5000         55000

gross salary of employee with eno 4: 44000
>>>
```

3. Create CSV file for product selling for 6 months and add only 5 record for 5 different product.

Prod_name	JAN	FEB	MAR	APR	MAY	JUN

Create Python script for following program: 1) Read data into DataFrame 2) Add columns and calculate total_sell, average_sell 3) Plot Total sell and average sell together on Line chart with proper legends, Titles and Lables. 4) Save the DataFrame to CSV named 'sell_analysis.csv'

Ans 3 :

```
import pandas as pd

import matplotlib.pyplot as plt

data={
   'prod_name':['A','B','C','D','E'],

   'JAN':[100,150,80,120,90],

   'FEB':[120,110,100,140,85]

  'MAR':[90,120,110,140,85],
   'APR':[110,140,90,125,105],

   'MAY':[130,100,120,110,115],

   'JUN':[105,130,95,135,125]

   }
df = pd.DataFrame(data)
 print('DataFram :') print(df)

 df['total_sell']=df.iloc[:,1:].sum(axis=1)

 df['average_sell']=df.iloc[:,1:].mean(axis=1) print(df)

 plt.figure(figsize=(10,6))

 plt.plot(df['prod_name'],df['total_sell'],label='Total

 Sell',marker='o')

 plt.plot(df['prod_name'],df['average_sell'],label='     Average

 Sell',marker='o')

  plt.xlabel('products')
```

```
plt.ylabel('amount')

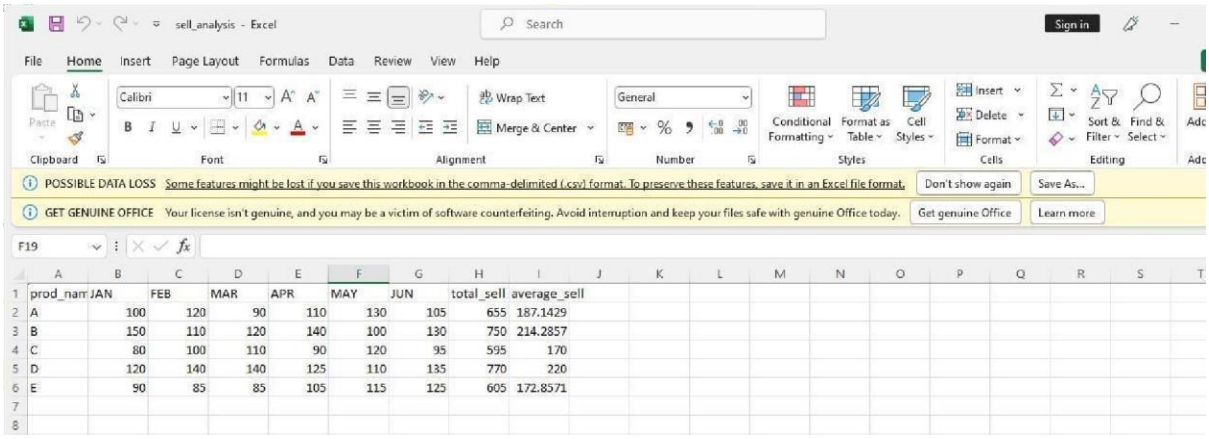
plt.title('product  sell analysis')

 plt.legend()

plt.grid(True)

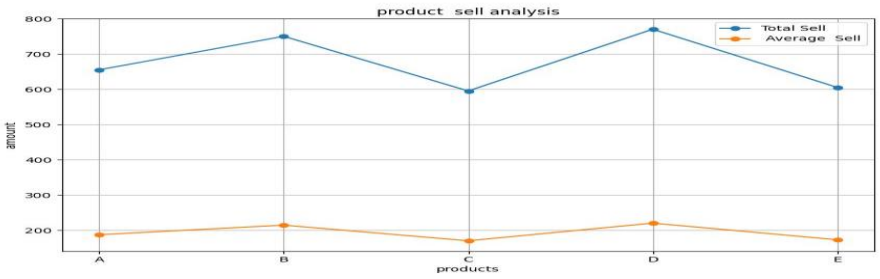
plt.show()

 df.to_csv('sell_analysis.csv',index=False)
```



prod_nam	JAN	FEB	MAR	APR	MAY	JUN	total_sell	average_sell
A	100	120	90	110	130	105	655	187.1429
B	150	110	120	140	100	130	750	214.2857
C	80	100	110	90	120	95	595	170
D	120	140	140	125	110	135	770	220
E	90	85	85	105	115	125	605	172.8571

```
============================== RESTART: D:/sql/j.py ==========================
DataFram :
  prod_name  JAN  FEB  MAR  APR  MAY  JUN
0         A  100  120   90  110  130  105
1         B  150  110  120  140  100  130
2         C   80  100  110   90  120   95
3         D  120  140  140  125  110  135
4         E   90   85   85  105  115  125
  prod_name  JAN  FEB  MAR  APR  MAY  JUN  total_sell  average_sell
0         A  100  120   90  110  130  105         655    187.142857
1         B  150  110  120  140  100  130         750    214.285714
2         C   80  100  110   90  120   95         595    170.000000
3         D  120  140  140  125  110  135         770    220.000000
4         E   90   85   85  105  115  125         605    172.857143
```



4. Write a phython script to do following on student (Rollno, Name, Sub 1, Sub 2, Sub 3, total) table: 1) Insert atleast 5 to 10 records. 2) Update the specific record value. 3) Delete the record specific record. 4) Display student detail who got highest total marks

Ans 4 :

```
import sqlite3
def create_table():
db=sqlite3.connect('student. db')
cur=db.cursor()
cur.execute('''create table student(
      rollno number, name text,   sub1
number,   sub2 number,  sub3 number,
total   number);''')
print('table created. ..')

create_table()
```

```
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/sqlite/sqlite-tools-win32-x86-3420000/q4.py
table created....
>>>
```

```
Import sqlite3 def insert_record():
db=sqlite3.connect('student.db')
cur=db.cursor()
```

```
cur.execute("insert into student
   values(1,'krish',90,95,85,270),(2,'veer',65,55,74,194),(3,'sujal
   ',96,74,55,225),(4,'janvi',85,76,98,259),(5,'ravi',80,45,66,191)
   ,(5,'aadi',96,84,66,246),(6,'naitik',65,48,90
   ,203),(7,'heet',40,25,31,96)")
 db.commit()
print('record inserted. ..')
insert_record()
```

```
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
======= RESTART: C:/sqlite/sqlite-tools-win32-x86-3420000/q4.py ======
record inserted....
>>>
```

```
SQLite version 3.42.0 2023-05-16 12:36:15
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
sqlite> .open student.db
sqlite> .table
sqlite> .mode box
sqlite> .table
student
sqlite> select * from student;
```

rollno	name	sub1	sub2	sub3	total
1	krish	90	95	85	270
2	veer	65	55	74	194
3	sujal	96	74	55	225
4	janvi	85	76	98	259
5	ravi	80	45	66	191
5	aadi	96	84	66	246
6	naitik	65	48	90	203
7	heet	40	25	31	96

```
sqlite>
```

```
Import sqlite3 def update_value():

db=sqlite3.connect('student.db')
```

```
 cur=db.cursor()

 cur.execute("update student set sub1=100 where

 rollno=1")

db.commit()
print('value updated... ')
 update_value()
```

```
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/sqlite/sqlite-tools-win32-x86-3420000/q4.py
value updated....
```

```
sqlite> select * from student;
```

rollno	name	sub1	sub2	sub3	total
1	krish	90	95	85	270
2	veer	65	55	74	194
3	sujal	96	74	55	225
4	janvi	85	76	98	259
5	ravi	80	45	66	191
5	aadi	96	84	66	246
6	naitik	65	48	90	203
7	heet	40	25	31	96

```
sqlite> select * from student;
```

rollno	name	sub1	sub2	sub3	total
1	krish	100	95	85	270
2	veer	65	55	74	194
3	sujal	96	74	55	225
4	janvi	85	76	98	259
5	ravi	80	45	66	191
5	aadi	96	84	66	246
6	naitik	65	48	90	203
7	heet	40	25	31	96

```
Import sqlite3 def delete_record():
 db=sqlite3.connect('student.db')                cur=db.cursor()
cur.execute("delete from student where name='sujal'")
db.commit()  print('record deleted... ')

 delete_record()
```

```
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/sqlite/sqlite-tools-win32-x86-3420000/q4.py
record deleted....
>>>
```

```
sqlite> select * from student;
```

rollno	name	sub1	sub2	sub3	total
1	krish	100	95	85	270
2	veer	65	55	74	194
4	janvi	85	76	98	259
5	ravi	80	45	66	191
5	aadi	96	84	66	246
6	naitik	65	48	90	203
7	heet	40	25	31	96

```
sqlite>
```

```
Import sqlite3
def highest_total(): db=sqlite3.connect('student.db')
    i=0
cur=db.cursor()
 cur.execute("select * from student order by total desc
limit 1")
 while
True:  record=cur.fetchone()  if record==None:
break i=i+1  print('\nstudent with highest total
marks
:')
print(record)

 highest_total()
```

```
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/sqlite/sqlite-tools-win32-x86-3420000/q4.py

student with highest total marks :
(1, 'krish', 100, 95, 85, 270)
>>>
```

5. Write Python Script to do followings on item.csv (Item_no, Item_name, Price, Qty, total) 1) Write item's detail in the item.csv file. Calculate total = price * Qty 2) Using data frame display item name and price whose price is between 1000 to 5000. 3) Display alternate records from item.csv file. 4) Display items whose price is minimum, maximum. 5) Sort the data according to item name wise. 6) Display items rows between 3th to 7th row. 7) Display last 6 rows.

```
import pandas as pd

df=pd.read_csv('item.csv') new_item={
  'Item_no':11,
  'Item_name':'mouse',
  'price':2000,
  'qty':10
  }
new_item['total']=new_item['price']* new_item['qty' ]
df=df._append(new_item,ignore_ind ex=True)
 df.to_csv('item.csv',index=False)
```

```
print('Item details added to item.csv')

item=df[(df['price'] >= 1000) & (df['price'] <=

5000)][['Item_name','price']]

print('\nitem with price between 1000 to 5000 :')

 print(item)

alternate_rec=df.iloc[::2] print('\nAlternate record:')

 print(alternate_rec)

min_price=df[df['price'] == df['price'].min()] max_price=df[df['price']

== df['price'].max()]

print('\nprint with minimum price :')
```

```
======= RESTART: D:/sqlite3/sqlite-tools-win32-x86-3420000/journal(5).py =======
Item details added to item.csv

items with price between 1000 to 5000:
   item_name  price
4   microwave   4500
5     blender   1500
7     tablets   5000
9     printer   1000
10      mouse   2000

Alternate records:
    item_no         item_name  price  qty   total
0         1            mobile  10000   20  200000
2         3  washing machine  12000   35  420000
4         5         microwave   4500   10   45000
6         7            laptop  25000   25  625000
8         9          computer  18000   25  450000
10       11             mouse   2000   10   20000

item with minimum price
   item_no item_name  price  qty  total
9       10   printer   1000   50  50000

item with maximum price
   item_no item_name  price  qty   total
6        7    laptop  25000   25  625000

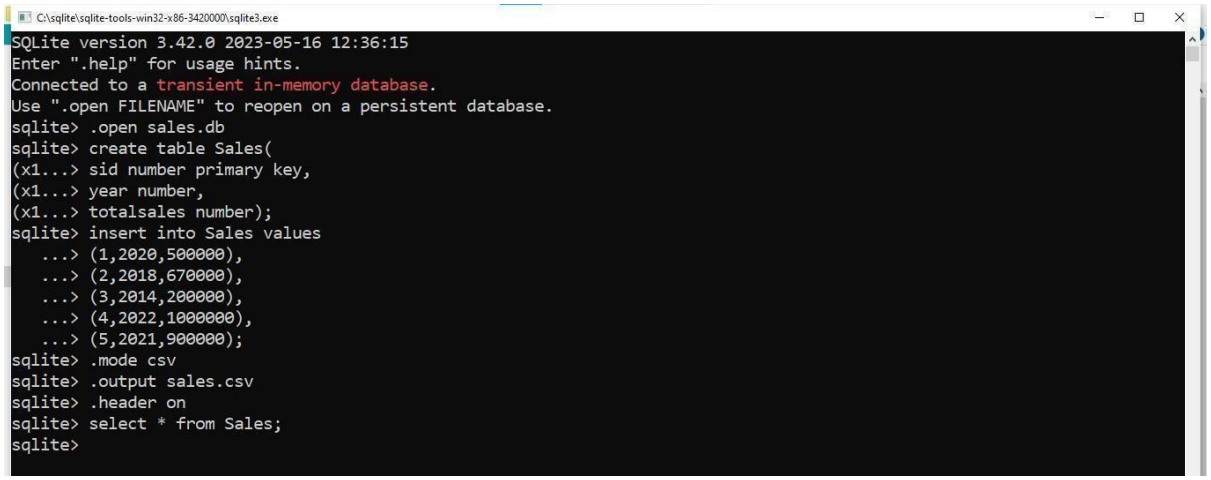
sorted by item names:
    item_no         item_name  price  qty   total
5         6           blender   1500   15   22500
8         9          computer  18000   25  450000
6         7            laptop  25000   25  625000
4         5         microwave   4500   10   45000
0         1            mobile  10000   20  200000
10       11             mouse   2000   10   20000
9        10           printer   1000   50   50000
3         4      refrigerator  20000   40  800000
7         8           tablets   5000  100  500000
1         2                tv  15000   10  150000
2         3  washing machine  12000   35  420000

item from 3rd to 7th row:
   item_no         item_name  price  qty   total
2        3  washing machine  12000   35  420000
3        4      refrigerator  20000   40  800000
4        5         microwave   4500   10   45000
5        6           blender   1500   15   22500
6        7            laptop  25000   25  625000

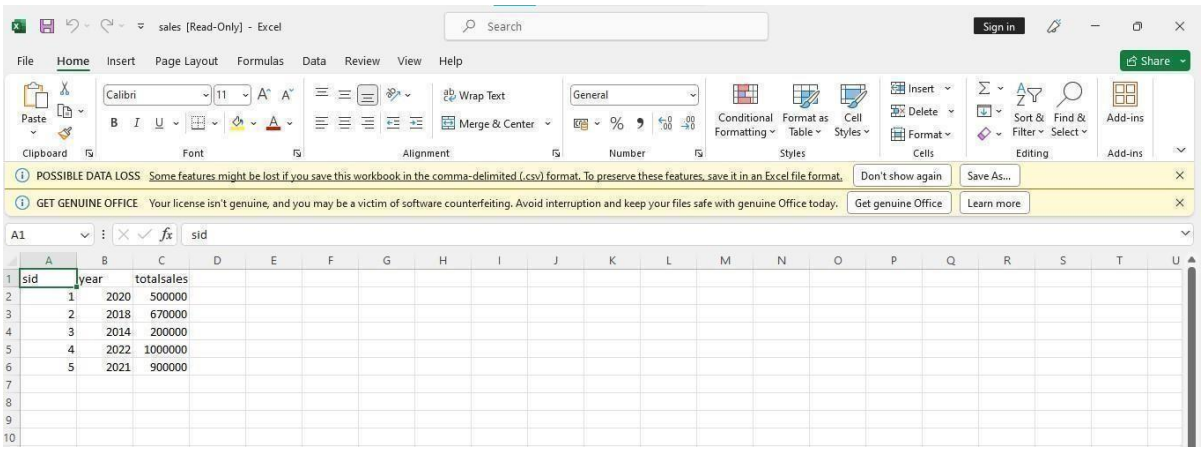
last 6 rows:
    item_no item_name  price  qty   total
5         6   blender   1500   15   22500
6         7    laptop  25000   25  625000
7         8   tablets   5000  100  500000
8         9  computer  18000   25  450000
9        10   printer   1000   50   50000
10       11     mouse   2000   10   20000
```

NAME: Kania Punit Chetan
ROLL-NO:276
DIV=D
SYBCA(Regular)-SEM-3

6. Sales (sid, year, totalsales) Create above table into a SQLite database with appropriate constraints. 1) Insert at least 5-10 records into the sales table 2) Export sales table data into sales.csv file. 3) Write a python scripts that read the sales.csv file and plot a bar chart that shows totalsales of the year. Also decorate the chart with appropriate title, lables, colours etc.

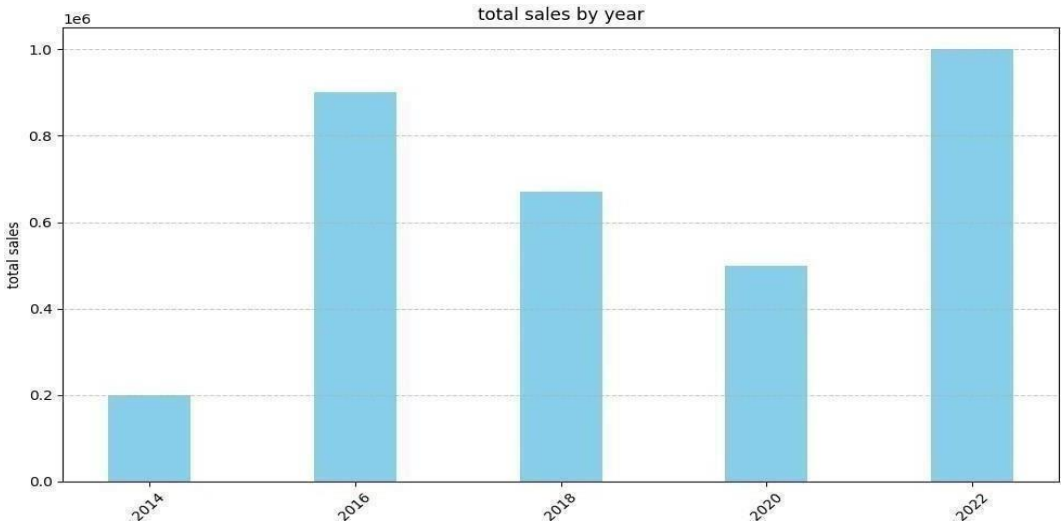







```
import csv import pandas as pd
import matplotlib.pyplot as  plt
with open('sales.csv', 'r') as file
df = csv.reader(file)
```

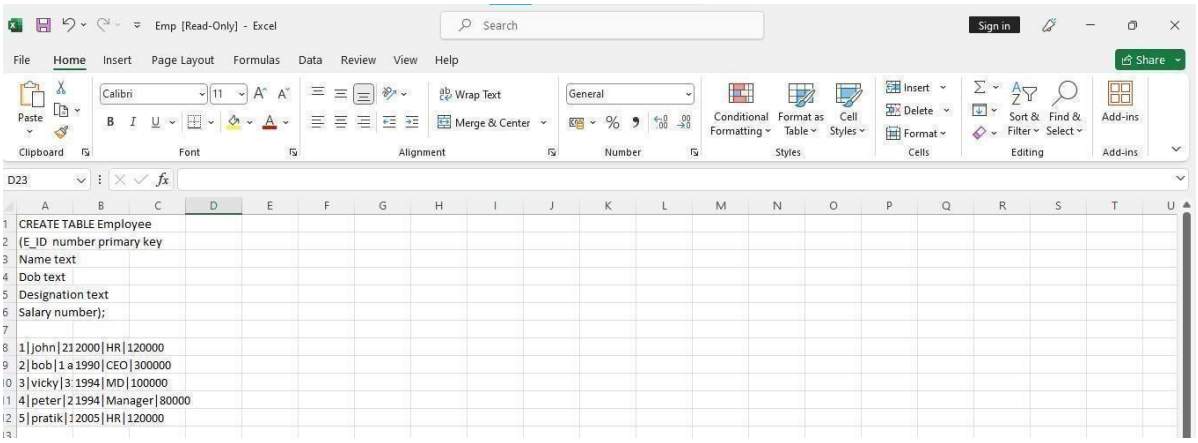
```
for row in df:
   print(row)
df=pd.read_csv('sales.cs  v')
 plt.figure(figsize=(10,6))
plt.bar(df['year'],df['totalsales'],colo r='skyblue')
plt.xlabel('year')
 plt.ylabel('total sales')
 plt.title('total  sales by year')
  plt.xticks(rotation=45)
  plt.grid(axis='y',linestyle='--',alpha=0.7)
  plt.tight_layout()
  plt.show()
```



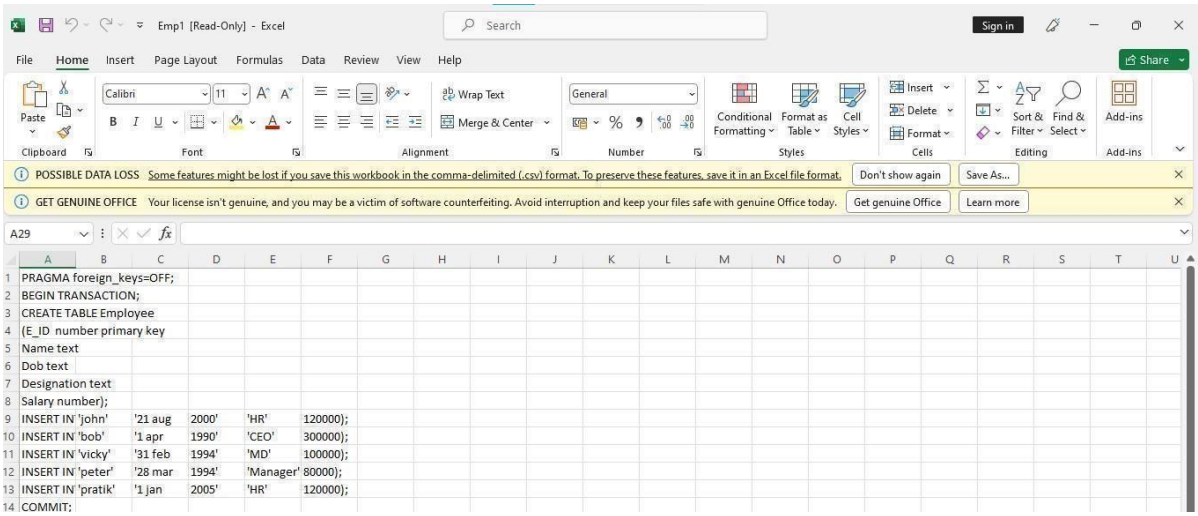
7. Create following table with appropriate constraints in Collage Database: Employee (E_ID, Name, Dob, Designation, Salary) a) Dump Employee table structure and data in Emp.csv file. b) Dump whole Database named College in Emp1.csv file.

```
SQLite version 3.42.0 2023-05-16 12:36:15
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
sqlite> .open collage
sqlite> create table Employee(
(x1...> E_ID number primary key,
(x1...> Name text,
(x1...> Dob text,
(x1...> Designation text,
(x1...> Salary number);
```

```
SQLite version 3.42.0 2023-05-16 12:36:15
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
sqlite> .open collage
sqlite> .output Emp.csv
sqlite> .schema Employee
sqlite> select * from Employee;
sqlite>
```

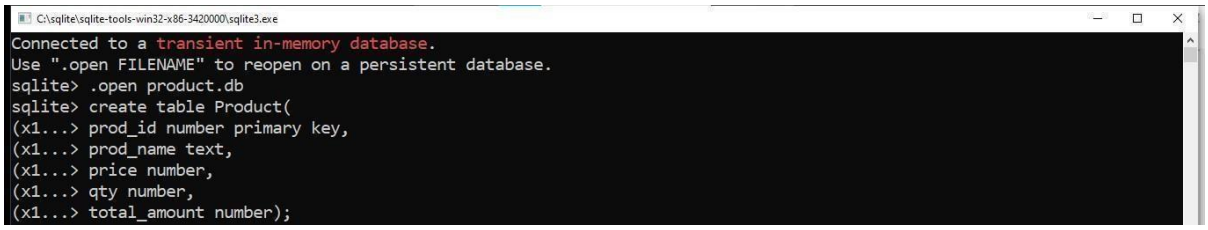


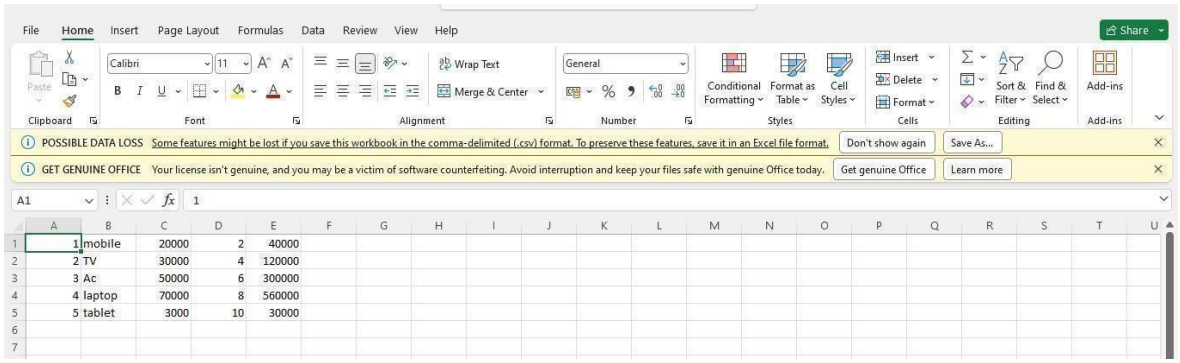

```
SQLite version 3.42.0 2023-05-16 12:36:15
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
sqlite> .open collage
sqlite> .tables
Employee
sqlite> .output Emp1.csv
sqlite> .dump
sqlite>
```

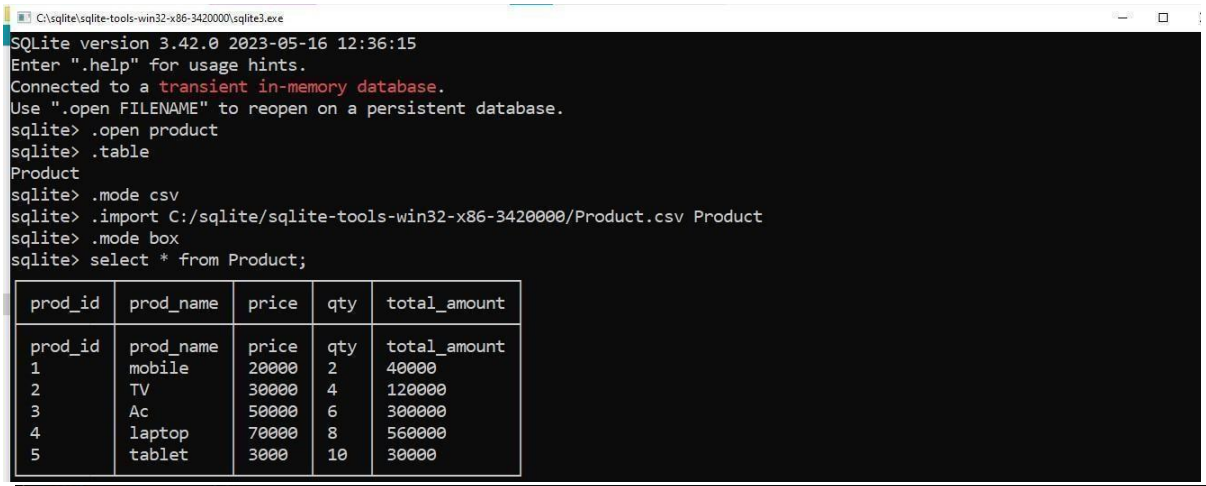


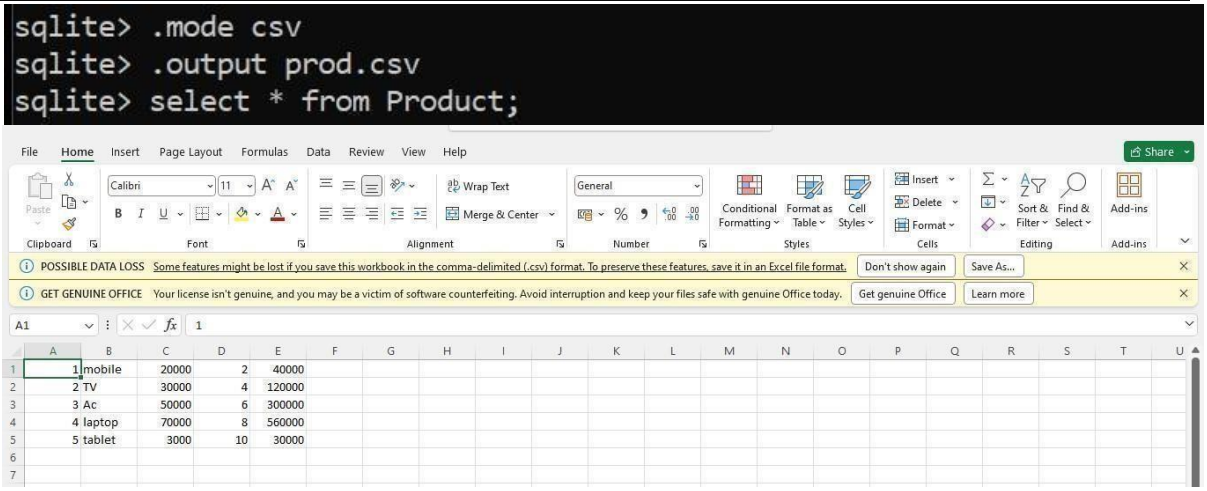
NAME: Kania Punit Chetan
ROLL-NO:276
DIV=D
SYBCA(Regular)-SEM-3

8. Create following table with appropriate Constraints: Product (prod_id , prod_name , price, qty,total_amount) 1) Import Product.csv file data into Product table. 2) Export Product table data into prod.csv file.

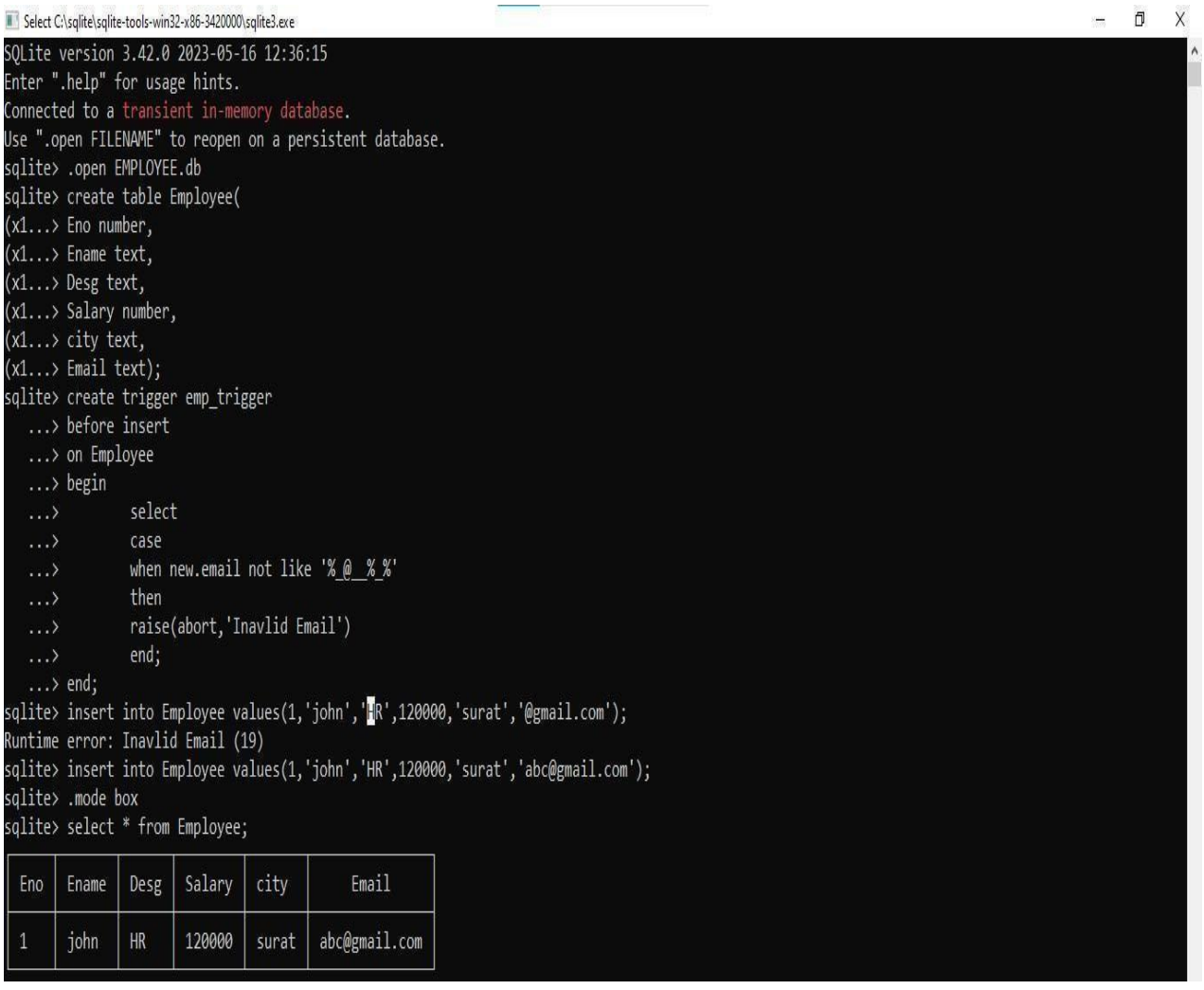








9. Employee(Eno number ,Ename text ,Desg text ,Salary number ,City text ,Email text) Write a SQL trigger named emp_trigger that is designed to execute before inserting records into the emp table. The trigger should perform the following action: 1) Check if the 'email' field in the newly inserted record follows a specific email address pattern. (example : abc@gmail.com)



10. Create COLLEGE database and perform following tasks.

a. Create following table using SQLite and then close the connection. Student (roll_no INTEGER Primary key, name text (20), city text (20), age INTEGER)

b. Insert 10 student records for Student table.

c. Add column course and display all records of student having city Navsari and course BCA using python cursor.

d. Dump sales tables query to sql file with name student_table.sql

e. Export the Student table to CSV file and display in Python IDLE.

```
import sqlite3
import csv

# a. Create the Student table
conn=sqlite3.connect('COLLEGE.db’)
cursor =
conn.cursor()

# Create Student table  cursor.execute('''CREATE TABLE IF NOT
EXISTS Student(roll_no INTEGER  PRIMARY KEY,
name  TEXT(20),city TEXT(20),age
INTEGER)''')

conn.commit()

# Insert 10 student records students =
[
   (1, 'John Doe', 'Navsari', 20),
   (2, 'Jane Doe', 'Surat', 22),
   (3, 'Mike Ross', 'Navsari', 21),
   (4, 'Rachel Zane', 'Ahmedabad', 23),
```

```
    (5, 'Harvey Specter', 'Navsari', 25),
    (6, 'Louis Litt', 'Mumbai', 24),
    (7, 'Donna Paulsen', 'Navsari', 22),
    (8, 'Jessica Pearson', 'Delhi', 27),
    (9, 'Sheila Sazs', 'Navsari', 26),
    (10, 'Benjamin Kim', 'Surat', 23)
]
cursor.executemany("INSERT INTO Student (roll_no, name, city, age) VALUES (?,
?, ?, ?)", students) conn.commit()

# c. Add column course and display students from
Navsari with course BCA cursor.execute("ALTER TABLE
Student ADD COLUMN course TEXT")  conn.commit()

# Update course for students
cursor.execute("UPDATE Student SET course =  'BCA' WHERE city =
'Navsari'") conn.commit()

# Fetch students from Navsari enrolled in BCA
cursor.execute("SELECT * FROM Student WHERE city = 'Navsari' AND course =
'BCA'") navsari_bca_students = cursor.fetchall()

# d. Dump Student table query to sql
file with
open('/mnt/data/student_table.s
ql', 'w') as f:      for line in
conn.iterdump():        f.write('%s\n' %
line)

# e. Export Student table to CSV
cursor.execute("SELECT * FROM Student")
all_students = cursor.fetchall()
```

```
csv_file_path =  '/mnt/data/student_table.csv'
with open(csv_file_path, 'w', newline='') as csv_file:
csv_writer = csv.writer(csv_file)
csv_writer.writerow([i[0]for i in cursor.description])  # Write
headers
 csv_writer.writerows(all_students)

conn.close()


# Show fetched Navsari BCA students navsari_bca_students
```

Output :

```
[ (1, 'John Doe', 'Navsari', 20, 'BCA'), (3, 'Mike Ross', 'Navsari', 21, 'BCA'), (5,
 'Harvey Specter', 'Navsari', 25, 'BCA'), (7, 'Donna Paulsen', 'Navsari', 22, 'BCA'),
 (9, 'Sheila Sazs', 'Navsari', 26, 'BCA')]
```

11. Create a table "tbl_Emp" in Database "MyDB.db" having fields like emp_no, emp_name, age, salary. a. Write python code to generate emp.csv file having all employees whose age > 25.

```
import sqlite3
import csv

# Connect to MyDB.db and create table tbl_Emp  conn
 = sqlite3.connect('MyDB.db')  cursor  =
conn.cursor()

# Create tbl_Emp table
cursor.execute('''CREATE TABLE IF NOT EXISTS tbl_Emp
(emp_no INTEGER PRIMARY KEY,
emp_name TEXT,
```

```
age INTEGER,
salary REAL)''')

# Insert employee records into tbl_Emp table
 employees = [
    (1, "Amit", 24, 30000),

    (2, "Punit", 27, 45000),

    (3, "Rahul", 30, 55000),

    (4, "Sneha", 22, 28000),

    (5, "Reema", 29, 60000)

 ]


cursor.executemany("INSERT INTO tbl_Emp (emp_no,
emp_name, age, salary)  VALUES (?, ?, ?, ?)", employees)
conn.commit()

# Fetch all employees with age > 25
cursor.execute("SELECT * FROM tbl_Emp WHERE age > 25")
employees_above_25 = cursor.fetchall()

# Export employees with age > 25 to emp.csv
csv_file_path = '/mnt/data/emp.csv' with
open(csv_file_path, 'w', newline='') as csv_file:
csv_writer = csv.writer(csv_file)
csv_writer.writerow([i[0] for i in cursor.description])
# Write headers
csv_writer.writerows(employees_above_25)
 conn.close()
```

Output employees with age > 25
employees_above_25

```
sqlite> select *from  tbl_emp;
1|Amit|24|30000.0
2|Punit|27|45000.0
3|Rahul|30|55000.0
4|Sneha|22|28000.0
5|Reema|29|60000.0
```

	A	B	C	D	E
1	emp_no	emp_nam	age	salary	
2	2	Punit	27	45000	
3	3	Rahul	30	55000	
4	5	Reema	29	60000	
5					

12. Create a SQLite database 'college' with table students (rollno, name, subject1, subjects, subject3) 1. insert 10 records 2. Write a python script that define function getdata (min, max). Here min define lower limit & max define upper limit. 3. Display all students records who score marks in given range in subject1

```
import sqlite3


# Step 1: Create database and table

con = sqlite3.connect("college.db")

cur = con.cursor()


cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS students (
    rollno INTEGER PRIMARY KEY,
    name TEXT,
    subject1 INTEGER,
    subject2 INTEGER,
    subject3 INTEGER
)
""")

# Step 2: Insert 10 records
records = [
    (1, "Amit", 45, 55, 60),
    (2, "Punit", 80, 70, 65),
    (3, "Riya", 62, 58, 71),
    (4, "Karan", 90, 85, 88),
    (5, "Sneha", 30, 40, 45),
    (6, "Meera", 50, 52, 48),
    (7, "Jay", 72, 68, 75),
```

```
    (8, "Nisha", 66, 70, 72),
    (9, "Tara", 39, 50, 49),
    (10, "Arjun", 95, 90, 92)
]

cur.executemany("INSERT OR REPLACE INTO students VALUES (?, ?, ?, ?, ?)", records)
con.commit()

# Step 3: Define function getdata(min, max)
def getdata(min_marks, max_marks):
    query = "SELECT * FROM students WHERE subject1 BETWEEN ? AND ?"
    cur.execute(query, (min_marks, max_marks))
    rows = cur.fetchall()

    # Display records
    print(f"\nStudents scoring between {min_marks} and {max_marks} in subject1:\n")
    print("RollNo\tName\tSubject1\tSubject2\tSubject3")
```

```
    print("-" * 50)
    for r in rows:
        print(f"{r[0]}\t{r[1]}\t{r[2]}\t\t{r[3]}\t\t{r[4]}")


# Example call
getdata(60, 90)


# Close connection
con.close()
```

```
sqlite> select *from students;
1|Amit|45|55|60
2|Punit|80|70|65
3|Riya|62|58|71
4|Karan|90|85|88
5|Sneha|30|40|45
6|Meera|50|52|48
7|Jay|72|68|75
8|Nisha|66|70|72
9|Tara|39|50|49
10|Arjun|95|90|92
```

```
Students scoring between 60 and 90 in subject1:

RollNo  Name    Subject1        Subject2        Subject3
--------------------------------------------------
2       Punit   80              70              65
3       Riya    62              58              71
4       Karan   90              85              88
7       Jay     72              68              75
8       Nisha   66              70              72
>
```

13. Write a Python code to implement following operation on Students table using SQLite module. Students (roll_no,s_name,subject 1,subject2,subject3). Write a menu driven python script perform following task . 1. insert records 2. Display all records using cursor. 3. Display Students detail who secures highest marks in subject1.

```
import sqlite3


con = sqlite3.connect("college.db")

cur = con.cursor()


cur.execute("""
CREATE TABLE IF NOT EXISTS Students (
   roll_no INTEGER PRIMARY KEY,
   s_name TEXT,
   subject1 INTEGER,
   subject2 INTEGER,
   subject3 INTEGER
)
""")
con.commit()
```

```
def insert_record():
    roll = int(input("Enter Roll No: "))
    name = input("Enter Student Name: ")
    s1 = int(input("Enter Subject1 Marks: "))
    s2 = int(input("Enter Subject2 Marks: "))
    s3 = int(input("Enter Subject3 Marks: "))

    cur.execute("INSERT INTO Students VALUES (?, ?, ?, ?, ?)", (roll, name, s1, s2, s3))
    con.commit()
    print("Record inserted successfully!\n")

def display_all():
    print("\nAll Student Records:\n")
    cur.execute("SELECT * FROM Students")
    rows = cur.fetchall()

    print("RollNo\tName\tSubject1\tSubject2\tSubject3")
```

```
    print("-" * 60)
    for r in rows:
        print(f"{r[0]}\t{r[1]}\t{r[2]}\t\t{r[3]}\t\t{r[4]}")
    print()

def highest_subject1():
    print("\nStudent(s) with Highest Marks in Subject1:\n")

    cur.execute("SELECT MAX(subject1) FROM Students")
    highest = cur.fetchone()[0]

    cur.execute("SELECT * FROM Students WHERE subject1 = ?", (highest,))
    rows = cur.fetchall()

    print("RollNo\tName\tSubject1\tSubject2\tSubject3")
    print("-" * 60)
    for r in rows:
        print(f"{r[0]}\t{r[1]}\t{r[2]}\t\t{r[3]}\t\t{r[4]}")
```

```
    print()


while True:
    print("===== STUDENT MANAGEMENT MENU =====")
    print("1. Insert Record")
    print("2. Display All Records")
    print("3. Display Student With Highest Marks in Subject1")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        insert_record()
    elif choice == 2:
        display_all()
    elif choice == 3:
        highest_subject1()
    elif choice == 4:
```

```
        print("Exiting Program...")
        break
    else:
        print("Invalid Choice! Try Again.\n")

con.close()
```

```
===== STUDENT MANAGEMENT MENU =====
1. Insert Record
2. Display All Records
3. Display Student With Highest Marks in Subject1
4. Exit
Enter your choice: 1
Enter Roll No: 11
Enter Student Name: puneet
Enter Subject1 Marks: 94
Enter Subject2 Marks: 89
Enter Subject3 Marks: 90
Record inserted successfully!

===== STUDENT MANAGEMENT MENU =====
1. Insert Record
2. Display All Records
3. Display Student With Highest Marks in Subject1
4. Exit
```

14. Write a python script to do following task. 1. Create Result Table (Rno, Student_name, IC, CPPM, DMA, Maths, CS) And Insert 10 Records. 2. Add Column Total Marks, Percentage in Result Table and Calculate Total and Percentage. 3. Display Student Name whose name start with 'A' and Percentage is between 40 and 50. 4. Display IC and CPPM Subject mark in ascending order. 5. Export all table data to csv named student_results.csv 6. Write a python script that read student_results.csv file and Plot a bar chart that shows Roll no and Total marks with proper Legends, titles and labels.

```
import sqlite3

import pandas as pd

import matplotlib.pyplot as plot

con=sqlite3.connect("clg.db")

cursor=con.cursor()


#cursor.execute("""create table result (Rno number, Student_name text,

  #        IC number, CPPM number, DMA number, Maths number, CS number);""")


'''cursor.execute("""insert into result values(1, 'Arjun', 45, 55, 60, 50, 65),

   (2, 'Amit', 40, 42, 45, 39, 50),

   (3, 'Rahul', 70, 65, 80, 75, 72),
```

```
    (4, 'Sneha', 60, 58, 62, 61, 59),

    (5, 'Karan', 30, 35, 40, 25, 38),

    (6, 'Meena', 55, 60, 58, 62, 57),

    (7, 'Riya', 80, 85, 78, 82, 90),

    (8, 'Anita', 38, 42, 40, 35, 45),

    (9, 'Vikas', 65, 68, 70, 72, 66),

    (10, 'Alok', 45, 48, 50, 52, 46); """)'''



#cursor.execute("alter table result add column total number")

#cursor.execute("alter table result add column persentage number")

#cursor.execute("update result set total=ic+cppm+dma+maths+cs")

cursor.execute("update result set
persentage=(ic+cppm+dma+maths+cs)*100/500")


df=pd.read_sql_query("select *from result", con)

print(df)
```

```
print(df[(df['Student_name'].str.startswith('A'))&(df['persentage'].between(40,
50))])


#print(df[df[['ic','cppm']].sort_values(by=['ic','cppm'])])

print(df[["IC", "CPPM"]].sort_values(by=["IC", "CPPM"]))

a=df.to_csv()

print(a)

with open ('result.csv','w') as f:

   df.to_csv(f)


df.plot.bar(x='rno',y='total',rot=15 ,title='bar chart')

plot.xlabel('rno')

plot.ylabel('total')

plot.show()

con.commit();
```

NAME: Kania Punit Chetan
ROLL-NO:276
DIV=D
SYBCA(Regular)-SEM-3

	Rno	Student_name	IC	CPPM	DMA	Maths	CS	total	persentage	percentage
0	1	Arjun	45	55	60	50	65	275	55.0	55
1	2	Amit	40	42	45	39	50	216	43.0	43
2	3	Rahul	70	65	80	75	72	362	72.0	72
3	4	Sneha	60	58	62	61	59	300	60.0	60
4	5	Karan	30	35	40	25	38	168	33.0	33
5	6	Meena	55	60	58	62	57	292	58.0	58
6	7	Riya	80	85	78	82	90	415	83.0	83
7	8	Anita	38	42	40	35	45	200	40.0	40
8	9	Vikas	65	68	70	72	66	341	68.0	68
9	10	Alok	45	48	50	52	46	241	48.0	48
10	1	Arjun	45	55	60	50	65	275	55.0	55
11	2	Amit	40	42	45	39	50	216	43.0	43
12	3	Rahul	70	65	80	75	72	362	72.0	72
13	4	Sneha	60	58	62	61	59	300	60.0	60
14	5	Karan	30	35	40	25	38	168	33.0	33
15	6	Meena	55	60	58	62	57	292	58.0	58
16	7	Riya	80	85	78	82	90	415	83.0	83
17	8	Anita	38	42	40	35	45	200	40.0	40
18	9	Vikas	65	68	70	72	66	341	68.0	68
19	10	Alok	45	48	50	52	46	241	48.0	48
20	1	Arjun	45	55	60	50	65	275	NaN	55

	Rno	Student_name	IC	CPPM	DMA	Maths	CS	total	persentage	percentage
1	2	Amit	40	42	45	39	50	216	43.0	43
7	8	Anita	38	42	40	35	45	200	40.0	40
9	10	Alok	45	48	50	52	46	241	48.0	48
11	2	Amit	40	42	45	39	50	216	43.0	43
17	8	Anita	38	42	40	35	45	200	40.0	40
19	10	Alok	45	48	50	52	46	241	48.0	48
21	2	Amit	40	42	45	39	50	216	NaN	43
27	8	Anita	38	42	40	35	45	200	NaN	40
29	10	Alok	45	48	50	52	46	241	NaN	48

	IC	CPPM
4	30	35
14	30	35
24	30	35
7	38	42
17	38	42
27	38	42
1	40	42
11	40	42
21	40	42

NAME: Kania Punit Chetan
ROLL-NO:276
DIV=D
SYBCA(Regular)-SEM-3

```
9    45    48
19   45    48
29   45    48
0    45    55
10   45    55
20   45    55
5    55    60
15   55    60
25   55    60
3    60    58
13   60    58
23   60    58
8    65    68
18   65    68
28   65    68
2    70    65
12   70    65
22   70    65
6    80    85
16   80    85
26   80    85
```

NAME: Kania Punit Chetan
ROLL-NO:276
DIV=D
SYBCA-SEM-3

